

Selective Knowledge Control for Continual Learning of GUI Agents Over Application Streams

Anonymous submission

Abstract

Continual learning is a crucial capability for Graphical User Interface (GUI) agents to adapt to evolving applications while retaining knowledge acquired from previous applications. Such application streams pose a challenging knowledge modeling problem: new applications often share underlying knowledge with past ones, yet also introduce specific knowledge that must not interfere with historical knowledge. In this paper, we propose activation-conditioned selective knowledge control, a lightweight method that achieves selective knowledge retention via neuron-level gradient manipulation. Our method maintains a compact historical knowledge state to protect highly activated MLP neurons that preserve previous knowledge. When a new application arrives, it performs real-time gradient surgery conditioned on forward activation. Concretely, the protected neurons are categorized into two types: unactivated neurons holding specific knowledge, whose gradients are truncated to prevent interference; and activated neurons holding shared knowledge, whose gradients are orthogonally projected to preserve stability while enabling adaptation. After each application stage, newly identified critical neurons are merged into the historical state for future learning. Empirical evaluations on multi-app sequential benchmark demonstrate that our method effectively mitigates catastrophic forgetting on prior applications while sustaining robust adaptation to new ones.

1 Introduction

Graphical User Interface (GUI) agents powered by Multimodal Large Language Models (MLLMs) have emerged as a pivotal paradigm for automating complex interactions on desktops or mobiles. Recent systems and benchmarks have shown rapid progress in visual grounding, action prediction, and long-horizon computer use (Cheng et al. 2024; Hong et al. 2024; Zhou et al. 2023; Rawles et al. 2024; Xie et al. 2025; Qin et al. 2025). Yet practical GUI-agent deployment rarely conforms to a fixed training paradigm. New applications with changed layouts, user-specific workflows, and emerging functions continually introduce new interaction tasks. This creates a natural continual learning setting for GUI agents: the agents must adapt to new applications

while retaining and reusing knowledge acquired from previous ones.

Continual learning of GUI agents over application streams raises a knowledge modeling problem. On one hand, a new application may rely on capabilities shared across diverse applications while also introducing application-specific behaviors that risk interfering with prior knowledge. For instance, diverse applications share general GUI operations, such as locating visual widgets, reading screen text, opening menus, and filling fields. On the other hand, they demand distinct workflows, such as building spreadsheet formulas, scheduling calendar events, or handling application-specific export configurations, as shown in Fig. 1. Consequently, continual GUI learning differs from traditional continual learning that only considers the trade-off between catastrophic forgetting and fast adaptation. Continual GUI learning requires selectively safeguarding application-specific knowledge while keeping shared knowledge adaptable and reusable.

Existing continual-learning strategies, however, often treat historical knowledge uniformly when imposing preservation constraints, making it difficult to distinguish application-specific knowledge from shared knowledge. Parameter freezing and importance regularization methods (Kirkpatrick et al. 2017) protect knowledge from previous applications, but they constrain protected components uniformly and may suppress updates to shared knowledge needed by a new application. Gradient surgery methods (Yu et al. 2020) further reduce destructive gradient interference by removing conflicting gradient components, but they do not distinguish reusable historical knowledge from knowledge that should be protected. Neuron-level protection methods are more fine-grained, but their fixed protected set still fails to distinguish protected and reusable historical knowledge (Serra et al. 2018; Masana, Tuytelaars, and van de Weijer 2021).

In this paper, we propose activation-conditioned selective knowledge control, a lightweight method that moves beyond uniform knowledge management by executing selective, neuron-level gradient-based control over historical knowledge. Given training data about a new application, the historical knowledge may play different roles: some knowledge is application-specific and should be preserved from unnecessary modification, while other knowledge is shared across applications and should remain adaptable and be further refined for the new application. The proposed method

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

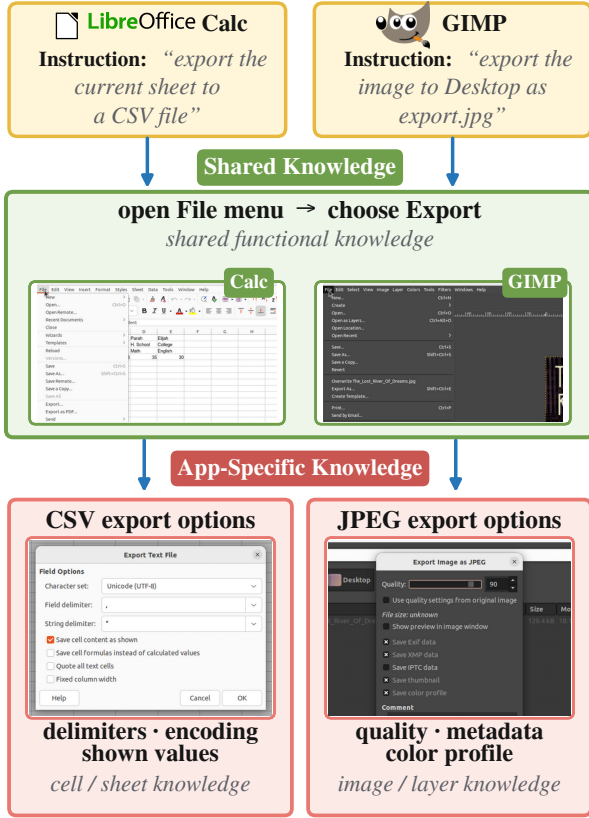


Figure 1: Example of shared and application-specific knowledge across GUI tasks. Exporting a spreadsheet in LibreOffice Calc and an image in GIMP shares the procedure of opening the File menu, selecting Export, and interacting with an export dialog, while the subsequent steps are application-specific.

therefore treats knowledge control as a dynamic distinction between protected historical knowledge and reusable shared knowledge, rather than as a static preservation rule.

Specifically, our method maintains a compact historical knowledge state to protect highly activated MLP neurons with their historical update directions, which preserve previous knowledge. During training on a new application, the proposed method performs activation-conditioned forward partitioning by registering a forward hook on MLP activations to collect neuron-level activation scores. During backward propagation, it performs neuron-level backward gradient surgery conditioned on the forward partition. For protected historical neurons that are not activated by the current update, their gradients are truncated to prevent unnecessary interference with application-specific historical knowledge. Meanwhile, for protected historical neurons that are activated by the current update, their gradients are projected onto the subspace orthogonal to cumulative historical update directions to preserve historical directions while allowing compatible adaptation. After each application training, a state update scheme is designed to merge newly identified important neu-

rons into the historical knowledge state for future learning. The resulting procedure is lightweight, requiring no historical trajectory replay, old-application gradient computation, or full application-specific checkpoints.

Our contributions are as follows:

- We propose activation-conditioned selective knowledge control for continual learning of GUI agents over application streams, which manages historical application knowledge selectively rather than uniformly by distinguishing application-specific knowledge from shared knowledge that should remain adaptable and reusable.
- We realize this method through neuron-level gradient operations conditioned on real-time activation: it truncates gradients of inactive protected neurons to prevent unnecessary interference and projects gradients of activated protected neurons orthogonal to cumulative historical update directions to allow compatible adaptation.
- Empirical evaluations on multi-app sequential benchmark demonstrate that the proposed method mitigates performance drops on previous applications while sustaining robust adaptation to new applications.

2 Related Work

2.1 GUI Agents

GUI agents aim to understand screen observations, follow natural language instructions, and execute low-level actions such as clicking, typing, and navigation in digital environments. Existing systems can be roughly grouped into structured agents that rely on HTML, accessibility trees, APIs, or other metadata (Zhou et al. 2023; Deng et al. 2023; Gur et al. 2023; Lai et al. 2024), visual agents that directly process screenshots with MLLMs (Cheng et al. 2024; You et al. 2024; Lu et al. 2024b; Xu et al. 2025), and hybrid agents that combine visual and structural signals (He et al. 2024; Gou et al. 2024; Wu et al. 2024; Qin et al. 2025). Concurrently, the training paradigm has evolved from supervised and data-centric fine-tuning on GUI demonstrations toward reinforcement fine-tuning from task outcomes or environment feedback (Hong et al. 2024; Lin et al. 2024; Chen et al. 2024; Lu et al. 2024a; Ou et al. 2024; Xu et al. 2024; Putta et al. 2024; Su et al. 2025). Meanwhile, benchmarks increasingly evaluate GUI agents across grounding, action prediction, multi-step task completion, and realistic computer-use workflows (Rawles et al. 2023, 2024; Xie et al. 2025; Li et al. 2025a; Nayak et al. 2025). However, most training pipelines assume a fixed training data distribution under a one-time post-training paradigm, while practical agents must continually adapt to changing applications, layouts, workflows, and tool APIs while retaining previously learned interaction abilities.

2.2 Continual Learning

Continual learning studies how a model can learn from a sequence of tasks while maintaining useful prior knowledge. Classical approaches preserve previous knowledge through importance regularization (Kirkpatrick et al. 2017) or replay-based constraints (Lopez-Paz and Ranzato 2017).

More closely related to our work are gradient-space methods that remove conflict gradients to avoid increasing losses on previous tasks (Lopez-Paz and Ranzato 2017; Farajtabar et al. 2020; Yu et al. 2020). Recent work has started to study continual learning in GUI and computer-use agents, focusing on domain and resolution shifts. GUI-AiF (Liu et al. 2026), GUI-AC (Lin et al. 2026), and ACuRL (Xue et al. 2026) improve continual adaptation by designing better rewards, optimization rules, or autonomous training data, but they do not directly control how new-application updates interact with historical GUI knowledge. The closest work is Agent-Dice (Wu et al. 2026), which addresses agent continual learning through post-hoc parameter fusion: it first trains on each application separately and then merges the resulting models. In contrast, our method treats continual learning as a gradient-control problem for a single sequentially fine-tuned model. Rather than merging models, it protects historical knowledge for the model while allowing shared functional neurons to adapt under an orthogonal constraint.

3 Method

3.1 Problem Setup and Formulation

We consider a GUI agent trained over an application stream with N applications. Each stage contains all training tasks and trajectories from one application and is indexed by $t = 1, \dots, N$. At stage t , the agent fine-tunes a policy model π_{θ_t} initialized from the previous stage $t-1$. Let $\mathcal{L}_t(\theta)$ denote the training objective over the current application stage. Standard sequential fine-tuning updates the model on stage t as follows,

$$\theta_t^{k+1} = \theta_t^k - \eta g_t^k, \quad g_t^k = \nabla_{\theta} \mathcal{L}_t(\theta_t^k), \quad (1)$$

where θ_t^k denotes the model parameters at optimization step k within stage t , and g_t^k is the current gradient. The effect of update in stage t on a historical objective $\mathcal{L}_i$ with $i < t$ can be approximated by a first-order Taylor expansion:

$$\begin{aligned} \mathcal{L}_i(\theta_t^{k+1}) &= \mathcal{L}_i(\theta_t^k - \eta g_t^k) \\ &\approx \mathcal{L}_i(\theta_t^k) + \langle \nabla_{\theta} \mathcal{L}_i(\theta_t^k), -\eta g_t^k \rangle \\ &= \mathcal{L}_i(\theta_t^k) - \eta \langle \nabla_{\theta} \mathcal{L}_i(\theta_t^k), g_t^k \rangle. \end{aligned} \quad (2)$$

Here, $\nabla_{\theta} \mathcal{L}_i(\theta_t^k)$ is the gradient of historical application i evaluated at the current parameters θ_t^k . This expression reveals the root cause of update interference: when $\langle \nabla_{\theta} \mathcal{L}_i(\theta_t^k), g_t^k \rangle < 0$, the resulting first-order change $-\eta \langle \nabla_{\theta} \mathcal{L}_i(\theta_t^k), g_t^k \rangle$ is positive, indicating that the current update conflicts with previous knowledge and increases the historical loss. Continual learning therefore aims to prevent such negative alignment and reduce the resulting increase in historical loss.

In our method, we explicitly distinguish between shared and application-specific knowledge across GUI applications. We argue that certain model neurons encode shared knowledge that should be continuously reinforced and updated along the application stream without increasing the loss on previous applications. Consequently, the update vector g_t^k for these neurons must be orthogonal to the historical loss gradient $\nabla_{\theta} \mathcal{L}_i(\theta_t^k)$, ensuring that $\langle \nabla_{\theta} \mathcal{L}_i(\theta_t^k), g_t^k \rangle = 0$. Conversely, for application-specific knowledge that requires no

modification, the update g_t^k on the corresponding neurons is set to zero to eliminate inter-application interference, naturally yielding $\langle \nabla_{\theta} \mathcal{L}_i(\theta_t^k), g_t^k \rangle = 0$. Finally, for neurons that do not store historical knowledge, we directly apply the unmodified gradient g_t^k for standard optimization. In this case, our approach continuously refines shared knowledge while preserving previously acquired application-specific knowledge, and at the same time enabling efficient learning of specific knowledge for new applications.

Specifically, we apply the method to MLP neurons in Transformer blocks. For clarity, let d_{ff} denote the MLP intermediate dimension, and let each neuron $j \in \{1, \dots, d_{\text{ff}}\}$ correspond to a parameter slice across the three MLP weight matrices: one column of $W^{\text{down}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, and one row of $W^{\text{gate}}, W^{\text{up}} \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$. At the beginning of stage t , the method loads a historical knowledge state $\mathcal{S}_t$ updated after previous stages:

$$\mathcal{S}_t = (C_t^{\text{hist}}, U_t^{\text{down}}, U_t^{\text{gate}}, U_t^{\text{up}}). \quad (3)$$

Here, $C_t^{\text{hist}} \subset \{1, \dots, d_{\text{ff}}\}$ tracks the set of neuron indices protected from past stages. For each projection $p \in \{\text{down}, \text{gate}, \text{up}\}$, $U_t^p = \{U_{j,t}^p\}_{j \in C_t^{\text{hist}}}$ maintains the corresponding historical update directions. Each element $U_{j,t}^p$ is an orthonormal basis matrix that spans the historical parameter-offset subspace for neuron j in projection p . The overall framework is shown in Fig. 2, and the exact stage-end state update from $\mathcal{S}_{t-1}$ to $\mathcal{S}_t$ is detailed in Section 3.4.

3.2 Activation-Conditioned Forward Partitioning

During the forward phase of training on stage t , the proposed method registers forward hooks on the inputs of MLP down projections in Transformer blocks and computes a runtime activation score for each neuron:

$$a_j = \frac{1}{BS} \sum_{b=1}^B \sum_{s=1}^S |h_{b,s,j}|, \quad (4)$$

where $h_{b,s,j}$ denotes the activation of neuron j for batch sample b and sequence position s . Rather than using a fixed numerical threshold, the proposed method selects the top $\lfloor \rho d_{\text{ff}} \rfloor$ activated neurons to form the current high-activation set:

$$C_t^{\text{cur}} = \text{Top}_{\rho}(\{a_j\}_{j=1}^{d_{\text{ff}}}), \quad (5)$$

where $\rho \in (0, 1]$ is the hyperparameter specifying the selection ratio.

The protected historical neurons are then partitioned according to whether they are reused by the current update:

$$C_t^{\text{share}} = C_t^{\text{hist}} \cap C_t^{\text{cur}}, \quad C_t^{\text{excl}} = C_t^{\text{hist}} \setminus C_t^{\text{cur}}. \quad (6)$$

This forward partitioning assigns neurons in C_t^{excl} to application-specific historical regions: they were important for previous application stages but are not actively used by the current update. In addition, the method assigns neurons in C_t^{share} to shared functional regions: they support historical application knowledge and are also activated by the current update, so they should remain trainable under a protection constraint.

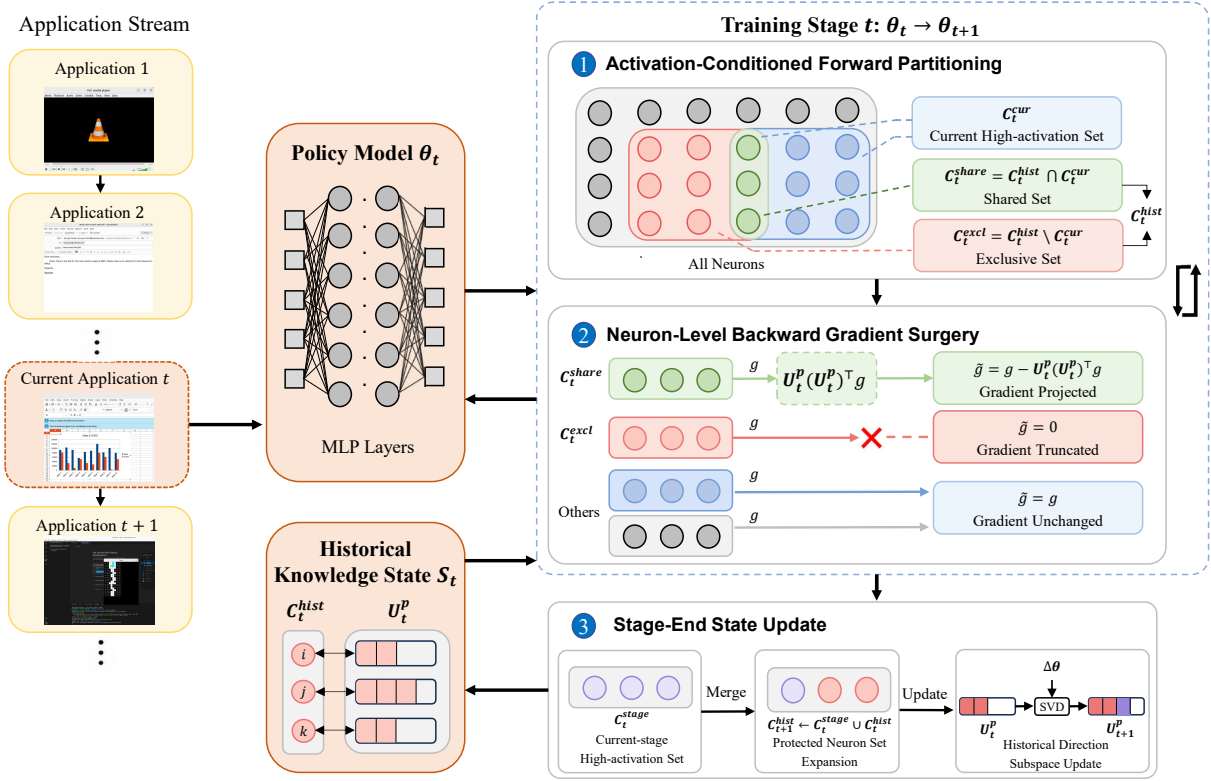


Figure 2: Overview of the proposed activation-conditioned selective knowledge control framework. Within each training stage, forward activations partition protected historical neurons into shared and application-specific subsets, and backward gradient surgery applies different update rules to these subsets. After the stage ends, the historical knowledge state is updated with newly identified important neurons and their historical direction subspaces.

3.3 Neuron-Level Backward Gradient Surgery

During the back-propagation phase, the proposed method modifies the gradients of the MLP projections at the neuron level before parameter updates. Let $\nabla_{W_p} \mathcal{L}_t$ denote the full gradient of the current loss $\mathcal{L}_t$ with respect to weight matrix W_p for $p \in \{\text{down}, \text{gate}, \text{up}\}$. For each neuron j , its corresponding gradient slice is given by

$$\begin{aligned} g_j^{\text{down}} &= (\nabla_{W^{\text{down}}} \mathcal{L}_t)[:, j], \\ g_j^{\text{gate}} &= ((\nabla_{W^{\text{gate}}} \mathcal{L}_t) [j, :])^\top, \\ g_j^{\text{up}} &= ((\nabla_{W^{\text{up}}} \mathcal{L}_t) [j, :])^\top. \end{aligned} \quad (7)$$

For application-specific historical neurons $j \in C_t^{\text{excl}}$, the proposed method blocks unnecessary current updates:

$$\tilde{g}_j^p = 0, \quad j \in C_t^{\text{excl}}. \quad (8)$$

This prevents the current update from modifying protected neurons that are not activated at the current stage.

For neurons $j \in C_t^{\text{share}}$ with shared knowledge, we project the current gradient g_j^p onto the orthogonal complement of the historical direction subspace spanned by $U_{j,t}^p$:

$$\tilde{g}_j^p = g_j^p - U_{j,t}^p (U_{j,t}^p)^\top g_j^p, \quad j \in C_t^{\text{share}}. \quad (9)$$

The remaining gradient is orthogonal to the historical update subspace and therefore preserves historical directions

while still allowing the shared neuron to adapt during new-application training.

For unprotected neurons $j \notin C_t^{\text{hist}}$, the proposed method leaves the gradient unchanged. The optimizer then updates the model using the modified gradients $\tilde{g}$.

3.4 Stage-End State Update

After completing stage t , we perform a stage-end state update that refreshes both the protected-neuron set C_t^{hist} and the historical direction subspaces U_t^p for $p \in \{\text{down}, \text{gate}, \text{up}\}$.

Protected Neuron Set Expansion. Unlike activation-conditioned forward partitioning, which uses runtime activations during training iterations, the stage-end update uses stage-level activation statistics collected over the completed application stage. Let $A_{j,t}$ denote the accumulated activation score of neuron j during stage t . The current stage-level important-neuron set is selected as

$$C_t^{\text{stage}} = \text{Top}_\rho \left(\{A_{j,t}\}_{j=1}^{d_{\text{ff}}} \right). \quad (10)$$

The historical protected set is then expanded by

$$C_{t+1}^{\text{hist}} = C_t^{\text{hist}} \cup C_t^{\text{stage}}. \quad (11)$$

Historical Direction Subspace Update. To update the historical direction subspaces, the stage-end update first evaluates the parameter displacement of each protected neuron $j \in C_{t+1}^{\text{hist}}$ induced by stage t . For each projection $p \in \{\text{down}, \text{gate}, \text{up}\}$, let $w_{j,t}^p$ and $w_{j,t+1}^p$ denote the parameter vector slice of neuron j before and after training on stage t , respectively. The stage-induced parameter offset is then defined as $\Delta w_{j,t+1}^p = w_{j,t+1}^p - w_{j,t}^p$.

We then collect its offset vectors across all stages in which this neuron was selected as stage-important to construct a historical offset matrix:

$$\mathbf{D}_{j,t+1}^p = [\Delta w_{j,r}^p]_{r \in \mathcal{I}_{j,t}}, \mathcal{I}_{j,t} = \{r \leq t \mid j \in C_r^{\text{stage}}\}. \quad (12)$$

We then perform truncated Singular Value Decomposition on $\mathbf{D}_{j,t+1}^p$ and keep the top r_{svd} left singular directions:

$$\mathbf{U}_{j,t+1}^p \approx U_{j,t+1}^p \Sigma_{j,t+1}^p (V_{j,t+1}^p)^\top. \quad (13)$$

The truncated left singular matrix $U_{j,t+1}^p$ forms a compact orthonormal basis for the dominant historical parameter-displacement subspace of neuron j , with at most r_{svd} directions retained. It is stored into the updated collection $U_{t+1}^p = \{U_{j,t+1}^p\}_{j \in C_{t+1}^{\text{hist}}}$.

The historical knowledge state is updated as

$$\mathcal{S}_{t+1} = (C_{t+1}^{\text{hist}}, U_{t+1}^{\text{down}}, U_{t+1}^{\text{gate}}, U_{t+1}^{\text{up}}). \quad (14)$$

This stage-end state update is performed between training stages, while online training only requires collecting current activations and modifying current gradients. Therefore, the proposed method does not replay historical trajectories, compute old-application gradients during new-application training, or maintain full application-specific checkpoints.

4 Experiments

4.1 Experimental Setup

Benchmark and Application Stream. We evaluate the proposed method on OSWorld using UI-TARS-1.5-7B as the base model for GUI agents. To simulate continual learning over applications, we construct an 8-stage application stream consisting of `vlc`, `Thunderbird`, `libreoffice_writer`, `libreoffice_impress`, `LibreOffice Calc`, `gimp`, `VSCoDe`, and `chrome`. Each stage t corresponds to an application in the stream. After training on stage t , the agent is evaluated on all seen applications up to stage t . The updated model and the historical knowledge state are then carried forward to initialize stage $t + 1$.

Implementation Details and Baseline. We build on the open-source DART-GUI infrastructure (Li et al. 2025b) and conduct RL training on OSWorld using UI-TARS-1.5-7B as the backbone model. We keep the released training and evaluation pipeline unchanged except for our selective knowledge control module. For our method, we set the neuron selection ratio as $\rho = 2\%$ and the truncated SVD rank as $r_{\text{svd}} = 4$. For comparison at each stage t , we construct a naive sequential fine-tuning baseline under the same infrastructure and optimization configuration, but without activation-conditioned selective knowledge control.

Evaluation Protocol. After each stage, we evaluate the agent three times on every application and report two groups of metrics: (1) **Success rate.** For each application, we report the average success rate over three evaluation runs. The Overall column reports the aggregate success rate over all seen applications, computed as the total number of successful evaluations divided by the total number of evaluation instances. (2) **Retention-adaptation metrics.** We use retained performance (RP), current-application performance (CP), and retention-adaptation harmonic score (RAH) to separate historical retention from current adaptation. At a given stage, for each application learned in an earlier stage, let s denote its current success rate and let b denote its best success rate before this stage. RP quantifies historical knowledge retention by penalizing performance drops relative to prior peak scores as $\text{RP} = 1 - \frac{\sum \max(b-s, 0)}{\sum b}$. CP is the success rate on the newly introduced application to measure current-stage adaptation performance, and RAH computes the harmonic mean of RP and CP to reflect the overall retention-adaptation balance.

4.2 Main Results

Table 1 reports the average success rates over all applications observed up to each training stage. The proposed method improves the overall performance at every comparable stage from Stage 2 to Stage 8. The gains are especially clear in the middle and later stages, where the overall success rate increases by 12.6, 7.1, and 10.6 percentage points at Stages 3, 6, and 7, respectively. These improvements indicate that activation-conditioned selective control helps the agent maintain stronger performance over the application stream instead of only adapting to the most recent application.

Table 2 further decomposes these gains into retention and current-application adaptation. The proposed method improves RP, CP, and RAH by 6.5, 9.1, and 8.3 percentage points on average, respectively. This shows that the method reduces performance degradation on previously learned applications while still allowing effective adaptation to the newly introduced application. A comparison with a replay baseline is provided in Appendix A.2.

4.3 Ablation Studies

Selective-control ablation. We compare four update-control settings to isolate the effect of activation-conditioned partitioning and neuron-level gradient surgery. The naive fine-tuning baseline leaves gradients unchanged for all protected neurons. The static projection variant removes activation-conditioned partitioning and projects gradients for all protected neurons. The static freezing variant truncates gradients for all protected neurons. Our method applies activation-aware selective control to protected neurons. Results are reported in Table 3. The baseline achieves a mean success rate of 37.6%. Static projection and static freezing improve the mean success rate to 41.5% and 40.3%, respectively. In contrast, our method reaches 47.1% on average and also achieves the best performance on Stage 2 (43.8%) and Stage 3 (50.3%), showing that activation-aware selective control better balances adaptation and preservation.

Method	Stage	VLC	Thunderbird	Writer	Impress	Calc	GIMP	VS Code	Chrome	Overall
Baseline	Stage 1	37.3	–	–	–	–	–	–	–	37.3
	Stage 2	33.3	42.2	–	–	–	–	–	–	37.5
	Stage 3	29.4	33.3	47.6	–	–	–	–	–	37.7
	Stage 4	33.3	42.2	46.0	37.9	–	–	–	–	39.5
	Stage 5	25.5	48.9	39.7	42.4	15.1	–	–	–	31.9
	Stage 6	23.5	53.3	33.3	45.5	13.5	57.7	–	–	35.5
	Stage 7	21.6	42.2	39.7	30.3	9.2	60.3	49.3	–	32.6
	Stage 8	31.4	53.3	38.1	40.9	13.5	65.4	50.7	31.7	37.3
Ours	Stage 1	37.3	–	–	–	–	–	–	–	37.3
	Stage 2	37.3 (+3.9)	51.1 (+8.9)	–	–	–	–	–	–	43.8 (+6.2)
	Stage 3	35.3 (+5.9)	57.8 (+24.4)	57.1 (+9.5)	–	–	–	–	–	50.3 (+12.6)
	Stage 4	37.3 (+3.9)	55.6 (+13.3)	54.0 (+7.9)	47.3 (+9.4)	–	–	–	–	48.3 (+8.8)
	Stage 5	37.3 (+11.8)	55.6 (+6.7)	55.6 (+15.9)	39.4 (-3.0)	24.3 (+9.2)	–	–	–	38.3 (+6.4)
	Stage 6	29.4 (+5.9)	51.1 (-2.2)	47.6 (+14.3)	53.0 (+7.6)	21.3 (+7.8)	62.8 (+5.1)	–	–	42.5 (+7.1)
	Stage 7	31.4 (+9.8)	44.4 (+2.2)	47.6 (+7.9)	41.2 (+10.9)	21.3 (+12.1)	70.5 (+10.3)	65.2 (+15.9)	–	43.3 (+10.6)
	Stage 8	35.3 (+3.9)	51.1 (-2.2)	47.6 (+9.5)	42.0 (+1.1)	21.4 (+8.0)	67.9 (+2.6)	63.8 (+13.0)	37.4 (+5.7)	42.7 (+5.4)

Table 1: Average success rates (%) over three runs on all seen applications after each training stage. Parentheses show signed percentage-point differences between ours and the baseline. Column-best results are bolded, and “–” denotes unavailable results.

Stage	Method	RP (%)	CP (%)	RAH (%)
Stage 2	Baseline	33.3	42.2	37.2
	Ours	37.3 (+4.0)	51.1 (+8.9)	43.1 (+5.9)
Stage 3	Baseline	31.4	47.6	37.8
	Ours	46.6 (+15.2)	57.1 (+9.5)	51.3 (+13.5)
Stage 4	Baseline	40.5	37.9	39.2
	Ours	44.8 (+4.3)	47.3 (+9.4)	46.0 (+6.9)
Stage 5	Baseline	39.1	15.1	21.8
	Ours	40.2 (+1.1)	24.3 (+9.2)	30.3 (+8.5)
Stage 6	Baseline	33.8	57.7	42.6
	Ours	40.5 (+6.7)	62.8 (+5.1)	49.2 (+6.6)
Stage 7	Baseline	33.9	49.3	40.2
	Ours	42.7 (+8.9)	65.2 (+15.9)	51.6 (+11.5)
Stage 8	Baseline	41.9	31.7	36.1
	Ours	47.0 (+5.1)	37.4 (+5.7)	41.7 (+5.6)
Mean	Baseline	36.3	40.2	36.4
	Ours	42.7 (+6.5)	49.3 (+9.1)	44.7 (+8.3)

Table 2: Retained performance, current-application performance, and their harmonic mean during Stages 2–8.

Hyperparameter analysis. We analyze the effect of two key hyperparameters: the selection ratio ρ used for activation-conditioned partitioning and the truncated SVD rank r_{svd} used for historical direction storage. Results are shown in Tables 4 and 5, respectively. The selection ratio table shows that $\rho = 2\%$ performs best on the three observed applications, with an overall success rate of 50.3%, while $\rho = 1\%$ and $\rho = 3\%$ reduce the overall success rate to 40.9% and 44.2%, respectively. The SVD-rank table shows that $r_{\text{svd}} = 4$ achieves the best overall success rate of 42.7%, whereas $r_{\text{svd}} = 2$ and $r_{\text{svd}} = 8$ yield lower performance. These results indicate that the proposed method benefits from a

Setting	Stage 2 (%)	Stage 3 (%)	Mean (%)
Baseline	37.5	37.7	37.6
Static Projection	42.7	40.3	41.5
Static Freezing	41.7	38.4	40.1
Ours	43.8	50.3	47.1

Table 3: Ablation studies on selective control with four update-control settings.

Selection Ratio	VLC	Thunderbird	Writer	Overall
$\rho = 1\%$	27.5	51.1	44.4	40.9
$\rho = 3\%$	33.3	48.9	48.3	44.2
Ours ($\rho = 2\%$)	35.3	57.8	57.1	50.3

Table 4: Hyperparameter analysis of the selection ratio ρ .

moderate selection ratio and a moderate number of retained historical update directions. Additional application-stream order ablations are provided in Appendix A.1.

4.4 Additional Analysis

Training overhead. We also measure the computational overhead introduced by activation-conditioned selective knowledge control during training. Table 6 compares the baseline and the proposed method under the same training configuration. Compared with the baseline, our method increases per-step training time from 4279.23s to 4324.00s, corresponding to a 1.0% increase. Token throughput decreases by 3.3%, and compute throughput decreases by 1.9%. These results support the lightweight design of the proposed method, which adds little per-step time cost while preserving most training throughput under the same training framework.

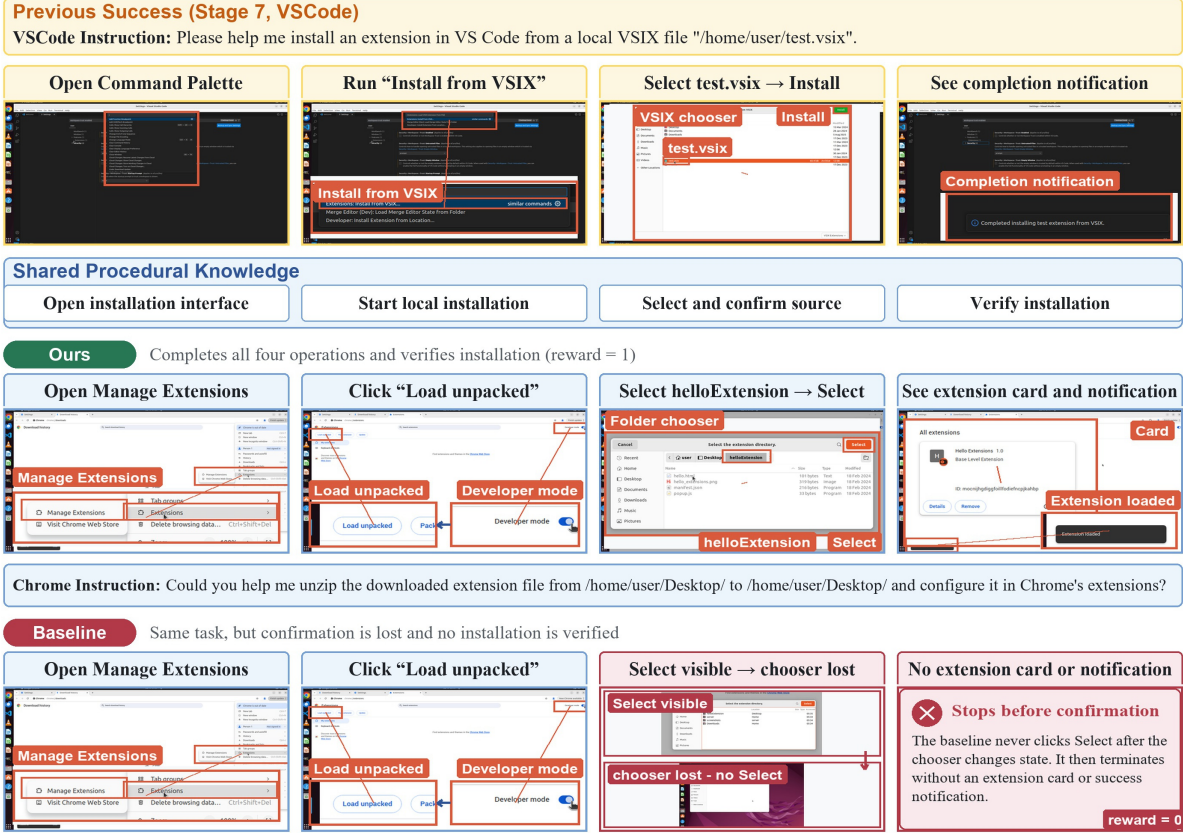


Figure 3: Shared procedural knowledge reuse on a Chrome task.

SVD Rank	$r_{\text{svd}} = 2$	$r_{\text{svd}} = 4$	$r_{\text{svd}} = 8$
Overall Success Rate (%)	38.5	42.7	38.7

Table 5: Hyperparameter analysis of the truncated SVD rank r_{svd} , reporting the overall success rate (%) at Stage 8 under different numbers of retained historical update directions.

Method	Time (s/step)	Tokens/s	TFLOPs
Baseline	4279.23	400.07	289.79
Ours	4324.00	386.73	284.30

Table 6: Training overhead evaluation. We report per-step training time, token throughput (Tokens/s), and compute throughput (TFLOPs) under the same training configuration.

Case study on shared knowledge reuse. We provide a qualitative case showing how shared procedural knowledge acquired at Stage 7 is reused at Stage 8. As shown in Figure 3, during the Stage 7 VSCode task, our method learns a general procedure from the successful *Install from VSIX* workflow: open the installation interface, start local installation, select and confirm the source, and verify the installation. At Stage 8, this shared knowledge is applied to the Chrome task through application-specific controls: our method opens

Manage Extensions, clicks *Load unpacked*, selects and confirms *helloExtension*, and verifies the extension card and success notification, thereby completing the task. In contrast, the baseline loses the confirmation control after the chooser changes state and terminates without confirming the source or verifying the installation, resulting in task failure. This case illustrates the transfer of shared procedural knowledge learned from the Stage 7 VSCode workflow to the Stage 8 Chrome task. Additional case studies on shared knowledge reuse and parameter-offset heatmap visualizations are provided in Appendix A.3 and Appendix A.4, respectively.

5 Conclusion

This paper has studied continual learning for GUI agents over application streams, where GUI agents must adapt to new applications while retaining and reusing knowledge for previous ones. By introducing activation-conditioned selective knowledge control, we move beyond uniform knowledge management through selective, neuron-level gradient-based control over historical knowledge, preserving application-specific knowledge while strengthening shared knowledge across applications. Empirical evaluations on multi-app sequential benchmark demonstrate that the proposed method mitigates performance drops on previous applications while sustaining robust adaptation to new applications.

References

- Chen, W.; Cui, J.; Hu, J.; Qin, Y.; Fang, J.; Zhao, Y.; Wang, C.; Liu, J.; Chen, G.; Huo, Y.; et al. 2024. Guicourse: From general vision language models to versatile gui agents. *arXiv preprint arXiv:2406.11317*.
- Cheng, K.; Sun, Q.; Chu, Y.; Xu, F.; YanTao, L.; Zhang, J.; and Wu, Z. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Annual Meeting of the Association for Computational Linguistics*, 9313–9332.
- Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36: 28091–28114.
- Farajtabar, M.; Azizan, N.; Mott, A.; and Li, A. 2020. Orthogonal Gradient Descent for Continual Learning. *arXiv preprint arXiv:1910.07104*.
- Gou, B.; Wang, R.; Zheng, B.; Xie, Y.; Chang, C.; Shu, Y.; Sun, H.; and Su, Y. 2024. Navigating the digital world as humans do: Universal visual grounding for gui agents. *arXiv preprint arXiv:2410.05243*.
- Gur, I.; Furuta, H.; Huang, A.; Safdari, M.; Matsuo, Y.; Eck, D.; and Faust, A. 2023. A real-world webagent with planning, long context understanding, and program synthesis. *arXiv preprint arXiv:2307.12856*.
- He, H.; Yao, W.; Ma, K.; Yu, W.; Dai, Y.; Zhang, H.; Lan, Z.; and Yu, D. 2024. WebVoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*.
- Hong, W.; Wang, W.; Lv, Q.; Xu, J.; Yu, W.; Ji, J.; Wang, Y.; Wang, Z.; Dong, Y.; Ding, M.; et al. 2024. Cogagent: A visual language model for gui agents. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 14281–14290.
- Kirkpatrick, J.; Pascanu, R.; Rabinowitz, N.; Veness, J.; Desjardins, G.; Rusu, A. A.; Milan, K.; Quan, J.; Ramalho, T.; Grabska-Barwinska, A.; et al. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13): 3521–3526.
- Lai, H.; Liu, X.; Iong, I. L.; Yao, S.; Chen, Y.; Shen, P.; Yu, H.; Zhang, H.; Zhang, X.; Dong, Y.; et al. 2024. AutoWebGLM: A Large Language Model-based Web Navigating Agent. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5295–5306.
- Li, K.; Meng, Z.; Lin, H.; Luo, Z.; Tian, Y.; Ma, J.; Huang, Z.; and Chua, T.-S. 2025a. Screenspot-pro: Gui grounding for professional high-resolution computer use. *arXiv preprint arXiv:2504.07981*.
- Li, P.; Hu, Z.; Shang, Z.; Wu, J.; Liu, Y.; Liu, H.; Gao, Z.; Shi, C.; Zhang, B.; Zhang, Z.; Shi, X.; Yu, Z.; Wu, Y.; Wu, X.; Jia, Y.; Xiang, L.; He, Z.; and Li, Q. 2025b. Efficient Multi-turn RL for GUI Agents via Decoupled Training and Adaptive Data Curation. *arXiv preprint arXiv:2509.23866*.
- Lin, C.; Feng, T.; Yuan, H.; Zhang, D.; Zhu, Y.; and Ou, Z. 2026. GUI-AC: Enhancing Continual Learning in GUI Agents. *arXiv preprint arXiv:2606.10522*.
- Lin, K. Q.; Li, L.; Gao, D.; Yang, Z.; Wu, S.; Bai, Z.; Lei, W.; Wang, L.; and Shou, M. Z. 2024. Showui: One vision-language-action model for gui visual agent. *arXiv preprint arXiv:2411.17465*.
- Liu, Z.; Kang, B.; Yuan, H.; Zhao, Z.; Li, W.; Zhu, Y.; and Feng, T. 2026. Continual GUI Agents. *arXiv preprint arXiv:2601.20732*.
- Lopez-Paz, D.; and Ranzato, M. 2017. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 6467–6476.
- Lu, Q.; Shao, W.; Liu, Z.; Meng, F.; Li, B.; Chen, B.; Huang, S.; Zhang, K.; Qiao, Y.; and Luo, P. 2024a. Gui odyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. *arXiv preprint arXiv:2406.08451*.
- Lu, Y.; Yang, J.; Shen, Y.; and Awadallah, A. 2024b. Omniparser for pure vision based gui agent. *arXiv preprint arXiv:2408.00203*.
- Masana, M.; Tuytelaars, T.; and van de Weijer, J. 2021. Ternary Feature Masks: Zero-Forgetting for Task-Incremental Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 3570–3579.
- Nayak, S.; Jian, X.; Lin, K. Q.; Rodriguez, J. A.; Kalsi, M.; Awal, R.; Chapados, N.; Özsü, M. T.; Agrawal, A.; Vazquez, D.; et al. 2025. Ui-vision: A desktop-centric gui benchmark for visual perception and interaction. *arXiv preprint arXiv:2503.15661*.
- Ou, T.; Xu, F. F.; Madaan, A.; Liu, J.; Lo, R.; Sridhar, A.; Sengupta, S.; Roth, D.; Neubig, G.; and Zhou, S. 2024. Synatra: Turning indirect knowledge into direct demonstrations for digital agents at scale. *arXiv preprint arXiv:2409.15637*.
- Putta, P.; Mills, E.; Garg, N.; Motwani, S.; Finn, C.; Garg, D.; and Rafailov, R. 2024. Agent q: Advanced reasoning and learning for autonomous ai agents. *arXiv preprint arXiv:2408.07199*.
- Qin, Y.; Ye, Y.; Fang, J.; Wang, H.; Liang, S.; Tian, S.; Zhang, J.; Li, J.; Li, Y.; Huang, S.; et al. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *arXiv preprint arXiv:2501.12326*.
- Rawles, C.; Clinckemaiellie, S.; Chang, Y.; Waltz, J.; Lau, G.; Fair, M.; Li, A.; Bishop, W.; Li, W.; Campbell-Ajala, F.; et al. 2024. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*.
- Rawles, C.; Li, A.; Rodriguez, D.; Riva, O.; and Lillicrap, T. 2023. Androidinthewild: A large-scale dataset for android device control. *Advances in Neural Information Processing Systems*, 36: 59708–59728.
- Serra, J.; Suris, D.; Miron, M.; and Karatzoglou, A. 2018. Overcoming Catastrophic Forgetting with Hard Attention to the Task. In *International Conference on Machine Learning*, 4548–4557.
- Su, H.; Sun, R.; Yoon, J.; Yin, P.; Yu, T.; and Arık, S. Ö. 2025. Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments. *arXiv preprint arXiv:2501.10893*.

Wu, Z.; Lou, X.; Ma, X.; Li, Y.; Liu, W.; Zhang, W.; Wang, J.; and Zhang, Z. 2026. Agent-Dice: Disentangling Knowledge Updates via Geometric Consensus for Agent Continual Learning. *arXiv preprint arXiv:2601.03641*.

Wu, Z.; Wu, Z.; Xu, F.; Wang, Y.; Sun, Q.; Jia, C.; Cheng, K.; Ding, Z.; Chen, L.; Liang, P. P.; et al. 2024. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218*.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Toh, J. H.; Cheng, Z.; Shin, D.; Lei, F.; et al. 2025. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37: 52040–52094.

Xu, Y.; Lu, D.; Shen, Z.; Wang, J.; Wang, Z.; Mao, Y.; Xiong, C.; and Yu, T. 2024. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. *arXiv preprint arXiv:2412.09605*.

Xu, Y.; Wang, Z.; Wang, J.; Lu, D.; Xie, T.; Saha, A.; Sahoo, D.; Yu, T.; and Xiong, C. 2025. Aguviz: Unified Pure Vision Agents for Autonomous GUI Interaction. In *International Conference on Machine Learning*.

Xue, T.; Liao, Z.; Shi, T.; Wang, Z.; Zhang, K.; Song, D.; Su, Y.; and Sun, H. 2026. Autonomous Continual Learning for Environment Adaptation of Computer-Use Agents. *arXiv preprint arXiv:2602.10356*.

You, K.; Zhang, H.; Schoop, E.; Weers, F.; Swearngin, A.; Nichols, J.; Yang, Y.; and Gan, Z. 2024. Ferret-ui: Grounded mobile ui understanding with multimodal llms. In *European Conference on Computer Vision*, 240–255. Springer.

Yu, T.; Kumar, S.; Gupta, A.; Levine, S.; Hausman, K.; and Finn, C. 2020. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*.

Zhou, S.; Xu, F. F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; Fried, D.; et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*.